

ASSIGNMENT 1

PERCEPTRONS

QUESTION 1

Design perceptrons for the following logic gates

1. AND 2. OR 3. NAND 4. NOR

SOLUTION

1. AND Gate:

X	Y	Output
0	0	0
0	1	0
1	0	0
1	1	1

```
t = 1
f = 0

learning_rate = 0.1

threshold = 1.5

inputs = [(0, 0), (0, 1), (1, 0), (1, 1)]
outputs = [0, 0, 0, 1]
bias = 1.5

weights = [0.5, 0.3]

def activation(weighted_sum):
    if weighted_sum < threshold:
        return f
    return t

new_weights = weights
# print(new_weights)

for idx, input in enumerate(inputs):
    weighted_sum = inputs[idx][0] * weights[0] + inputs[idx][1] * weights[1] + bias
    if activation(weighted_sum) != outputs[idx]:
        # update weights
        for i in range(len(weights)):
            # print(i)
            new_weights[i] += (
                learning_rate
                * (outputs[idx] - activation(weighted_sum))
```

```

        * inputs[idx][i]
    )

    bias += learning_rate * (outputs[idx] - activation(weighted_sum)) * 1
    if new_weights != weights:
        weights = new_weights
        # new_weights = []

    print(f"Epoch {idx}: {''.join(str(weights))}, \tnew bias: {bias}")

```

2. OR Gate

X	Y	Output
0	0	0
0	1	1
1	0	1
1	1	1

```

t = 1
f = 0

learning_rate = 0.1

threshold = 1.5

inputs = [(0, 0), (0, 1), (1, 0), (1, 1)]
outputs = [0, 1, 1, 1]
bias = 1.5

weights = [0.5, 0.3]

def activation(weighted_sum):
    if weighted_sum < threshold:
        return f
    return t

new_weights = weights

for idx, input in enumerate(inputs):
    weighted_sum = inputs[idx][0] * weights[0] + inputs[idx][1] * weights[1] + bias
    if activation(weighted_sum) != outputs[idx]:
        for i in range(len(weights)):
            new_weights[i] += (
                learning_rate
                * (outputs[idx] - activation(weighted_sum))
                * inputs[idx][i]
            )

        bias += learning_rate * (outputs[idx] - activation(weighted_sum)) * 1
    if new_weights != weights:

```

```

weights = new_weights

print(f"Epoch {idx}: {''.join(str(weights))}, \tnew bias: {bias}")

```

3. NAND Gate

X	Y	Output
0	0	1
0	1	1
1	0	1
1	1	0

```

t = 1
f = 0

learning_rate = 0.1

threshold = 1.5

inputs = [(0, 0), (0, 1), (1, 0), (1, 1)]
outputs = [1, 1, 1, 0]
bias = 1.5

weights = [0.5, 0.3]

def activation(weighted_sum):
    if weighted_sum < threshold:
        return f
    return t

new_weights = weights

for idx, input in enumerate(inputs):
    weighted_sum = inputs[idx][0] * weights[0] + inputs[idx][1] * weights[1] + bias
    if activation(weighted_sum) != outputs[idx]:
        for i in range(len(weights)):
            new_weights[i] += (
                learning_rate
                * (outputs[idx] - activation(weighted_sum))
                * inputs[idx][i]
            )

        bias += learning_rate * (outputs[idx] - activation(weighted_sum)) * 1
    if new_weights != weights:
        weights = new_weights

print(f"Epoch {idx}: {''.join(str(weights))}, \tnew bias: {bias}")

```

4. NOR Gate

X	Y	Output
0	0	1
0	1	0
1	0	0
1	1	0

```

t = 1
f = 0

learning_rate = 0.1

threshold = 1.5

inputs = [(0, 0), (0, 1), (1, 0), (1, 1)]
outputs = [1, 0, 0, 0]
bias = 1.5

weights = [0.5, 0.3]

def activation(weighted_sum):
    if weighted_sum < threshold:
        return f
    return t

new_weights = weights

for idx, input in enumerate(inputs):
    weighted_sum = inputs[idx][0] * weights[0] + inputs[idx][1] * weights[1] + bias
    if activation(weighted_sum) != outputs[idx]:
        for i in range(len(weights)):
            new_weights[i] += (
                learning_rate
                * (outputs[idx] - activation(weighted_sum))
                * inputs[idx][i]
            )

        bias += learning_rate * (outputs[idx] - activation(weighted_sum)) * 1
    if new_weights != weights:
        weights = new_weights

print(f"Epoch {idx}: {' '.join(str(weights))}, \tnew bias: {bias}")

```